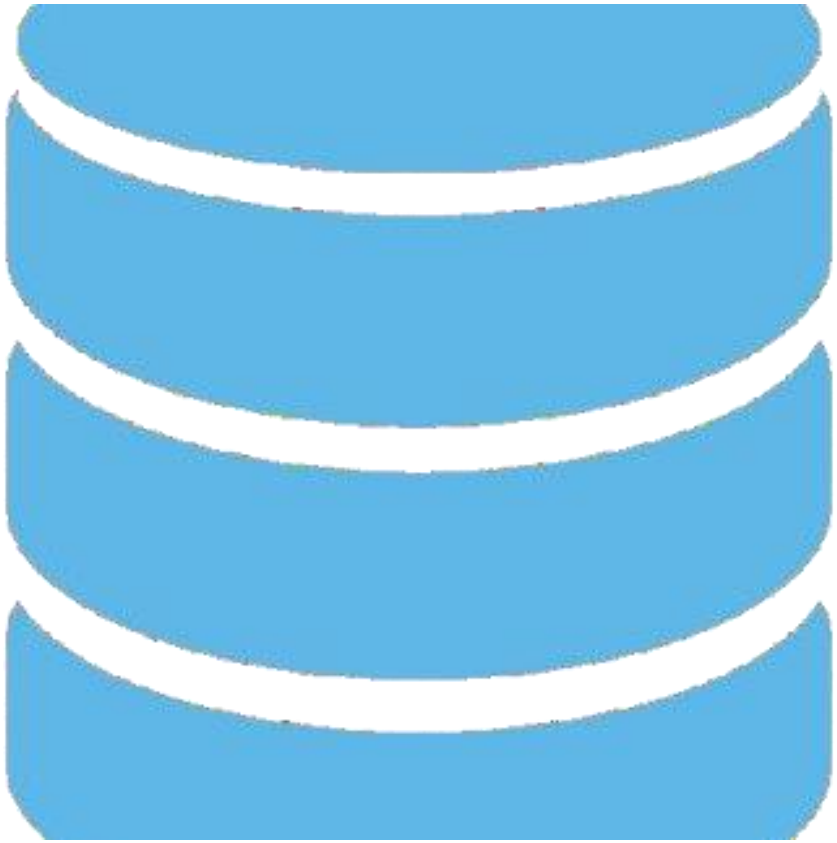
Requêtes SQL et accès aux données

PAR *HAROUNA KANE – Groupe SR3 SIMPLON*



SOMMAIRE

1. Exercices et atelier final
2. Transactions
3. Accès aux données : connexion, repository, ORM
4. Tests des requêtes et du code
5. SQL de définition (DDL)
6. SQL de manipulation (DML)
7. Jointures
8. Fonctions temporelles
9. Fonctions d’agrégation
10. Sous-requêtes

CRÉATION DE TABLES VIA MYSQL



CRÉATION DE TABLES VIA MYSQL

Créer une table

```sql
CREATE TABLE auteur (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nom VARCHAR(100),
  pays VARCHAR(50)
);
```

Nom de la table : *auteur*

Colonne/attribut : *id, nom, pays*

Identifiant : *id*

CRÉATION DE TABLES VIA MYSQL

Création d'une table via une interface graphique

Les interfaces graphiques comme **MySQL Workbench**, **phpMyAdmin**, **Oracle** ou **DBeaver** offrent des outils visuels permettant de créer et gérer des bases de données sans écrire de code SQL.

Avantages :

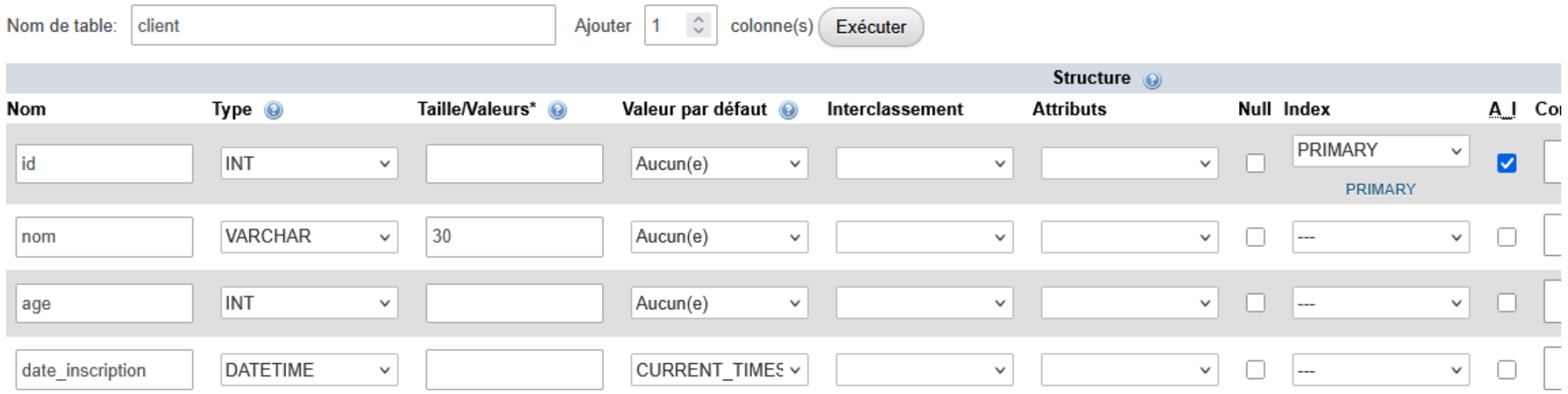
- Interface intuitive, facile à prendre en main
- Ajout de colonnes via des menus déroulants
- Définition rapide des types de données (INT, VARCHAR, DATE…)
- Cases à cocher pour définir : **clés primaires**, **contraintes**, **auto-incrément**, etc.
- Aperçu direct du script SQL généré

CRÉATION DE TABLES VIA MYSQL

Création d'une table via une interface graphique







MODIFICATION DE LA STRUCTURE D'UNE TABLE

Modifier une table – ALTER TABLE

Une fois une table créée, il est fréquent de devoir la modifier :

- ✓ Ajouter une colonne
- ✓ Supprimer une colonne
- ✓ Modifier un type de données
- ✓ Renommer une colonne
- ✓ Ajouter une contrainte

MODIFICATION DE LA STRUCTURE D'UNE TABLE

Ajouter une colonne

```sql
ALTER TABLE utilisateur ADD email VARCHAR(100);
```

Supprimer une colonne

```sql
ALTER TABLE utilisateur DROP COLUMN email;
```

Modifier une colonne

```sql
ALTER TABLE utilisateur MODIFY nom VARCHAR(150);
```

Renommer une colonne

```sql
ALTER TABLE utilisateur CHANGE nom nom_utilisateur VARCHAR(150);
```

Ajouter une clé étrangère

```sql
ALTER TABLE commande ADD CONSTRAINT fk_utilisateur FOREIGN KEY(id_utilisateur)
REFERENCES utilisateur(id_utilisateur);
```

INSÉRER DES DONNÉES – INSERT INTO

La commande `INSERT` – Ajouter de nouvelles données

Syntaxe simple :

```
INSERT INTO nom_table (colonne1, colonne2, ...)
VALUES (valeur1, valeur2, ...);
```

Exemple concret :

```
INSERT INTO internaute (prenom, nom, email, pays, code_region)
VALUES ('Jens', 'Fall', 'jfall@example.com', 'Sénégal', 'SN-DK');
```

Cas où toutes les colonnes sont remplies dans l'ordre :

```
INSERT INTO genre VALUES (11, 'Historique');
```

INSÉRER DES DONNÉES – INSERT INTO

La commande `INSERT` – Ajouter de nouvelles données

Insérer plusieurs lignes en une seule commande :

```sql
INSERT INTO genre (id, nom) VALUES
(12, 'Musical'),
(13, 'Western'),
(14, 'Documentaire');
```

⚠️ **Bonnes pratiques :**

- Spécifiez toujours les colonnes pour éviter les erreurs si la structure de la table change.
- Faites attention aux contraintes (clé primaire, clé étrangère, NOT NULL, etc.)

SELECTIONNER DES DONNÉES - SELECT

La commande `SELECT` – Extraire les données d'une base

Objectif :

Apprendre à interroger une base de données pour afficher des informations précises.

Syntaxe de base

```
SELECT colonne1, colonne2 FROM nom_table;
```

SELECTIONNER DES DONNÉES - SELECT

La commande SELECT – Extraire les données d'une base

Exemples simples :

```
SELECT * FROM film;
```

titre	annee	id_realisateur	resume	code_pays
365 jours : Au lendemain	2022	1192152	On retrouve Laura et Massimo dans une histoire plu...	PL
À contre-sens	2023	1739102	Noah doit quitter sa ville, son petit ami et ses a...	ES
À contre-sens 2	2024	1739102	La relation entre Noah et Nick semble plus forte q...	ES
À contre-sens : Londres	2025	2442185	Noah, 18 ans, quitte l'Amérique pour s'installer à...	EN
A Working Man	2025	19769	Levon Cade est un criminel repenti. Aujourd'hui,	EN

Liste tous les films.

SELECTIONNER DES DONNÉES - SELECT

La commande `SELECT` – Extraire les données d'une base

Les opérateurs : filtrer les données

Opérateurs de comparaison

Opérateur	Signification
=	Égal
<	Inférieur
<=	Inférieur ou égal
>	Supérieur
>=	Supérieur ou égal
<> ou !=	Différent
IS NULL	aucune valeur renseignée
IS NOT NULL	une valeur est renseignée

Opérateurs logiques

Opérateur	Symbole	Signification
AND	&&	Et
OR	||	Ou
NOT	!	non/pas

SELECTIONNER DES DONNÉES - SELECT

La commande SELECT – Extraire les données d'une base

Utiliser des filtres – WHERE

```
SELECT nom_colonne FROM nom_table WHERE [condition]
```

Choisis les films dont l'année est supérieure à 2000.

```
SELECT titre, annee FROM film WHERE annee >= 2000;
```

Choisis les films dont l'année est supérieure à 2000 et le code pays est égal à US.

```
SELECT * FROM film WHERE annee >= 2000 AND code_pays = 'US';
```

SELECTIONNER DES DONNÉES - SELECT

La commande SELECT – Extraire les données d'une base

Éviter les doublons – DISTINCT

```
SELECT DISTINCT code_pays FROM film;
```

Liste les différents codes pays de la table film.

Limiter le nombre de résultats – LIMIT

```
SELECT * FROM film LIMIT 10;
```

Sélectionne les 10 premiers films.

SELECTIONNER DES DONNÉES - SELECT

Trier et filtrer – ORDER BY, LIKE, BETWEEN, IN

Trier les résultats – ORDER BY (ASC par défaut, DESC pour décroissant)

```
SELECT titre, annee FROM film ORDER BY annee DESC, titre ASC;
```

Rechercher un motif – LIKE (% = plusieurs caractères, _ = un seul)

```
SELECT titre FROM film WHERE titre LIKE 'Le%';
```

Intervalle et liste – BETWEEN, IN

```
SELECT titre FROM film WHERE annee BETWEEN 2000 AND 2010;
SELECT titre FROM film WHERE code_pays IN ('FR', 'US');
```

Important : LIMIT n’a de sens qu’avec ORDER BY, sinon l’ordre n’est pas garanti.

MODIFIER DES DONNÉES - UPDATE

La commande `UPDATE` – Mettre à jour des données

Objectif :
Savoir modifier les valeurs d'une ou plusieurs colonnes d'un ou plusieurs enregistrements.

Syntaxe général :

```
UPDATE nom_table
SET colonne1 = nouvelle_valeur1, colonne2 =
nouvelle_valeur2
WHERE condition;
```

⚠️ **Attention :**
Toujours utiliser `WHERE` !
Sans condition, toutes les lignes seront modifiées.

MODIFIER DES DONNÉES - UPDATE

La commande UPDATE – Mettre à jour des données

Exemple concret :

```
UPDATE internaute
SET nom = 'Diop', prenom = 'Moussa'
WHERE id = 15;
```

Exemple de mise à jour multiple :

```
UPDATE film
SET code_pays = 'FR', annee = 2023
WHERE titre = 'Avatar';
```

SUPPIMER DES DONNÉES - DELETE

La commande DELETE – Supprimer des enregistrements

Retirer une ou plusieurs lignes d'une table, selon une ou plusieurs conditions.

Syntaxe de base :

```
DELETE FROM nom_table
WHERE condition;
```

Exemple concret :

```
DELETE FROM notation WHERE id_film = 12 AND note = 2;
```

Supprime les notes égales à 2 du film d'identifiant 12.

SUPPIMER DES DONNÉES - DELETE

La commande `DELETE` – Supprimer des enregistrements

⚠️ **Important :**

- **Toujours utiliser `WHERE`** pour éviter de supprimer toutes les données !
- Sans clause `WHERE`, **toutes les lignes** de la table seront supprimées :

```
DELETE FROM genre;
-- Supprime tous les genres
```

Alternative avec TRUNCATE (supprimer tous les enregistrements + réinitialiser l'auto-incrément) :

```
TRUNCATE TABLE film;
```

Vide la table et réinitialise l'auto-incrément. TRUNCATE est une commande DDL : elle ne peut pas être annulée.

Transactions : tout ou rien

Une transaction regroupe plusieurs requêtes : toutes réussissent (COMMIT) ou aucune n’est appliquée (ROLLBACK).

```
START TRANSACTION;
DELETE FROM inscription WHERE id_apprenant = 3 AND id_session = 1;
INSERT INTO inscription (id_apprenant, id_session, date_inscription)
VALUES (3, 2, CURDATE());
COMMIT;   -- ou ROLLBACK; en cas d'erreur
```

Propriétés ACID : Atomicité, Cohérence, Isolation, Durabilité.

Sous MySQL, le moteur InnoDB est nécessaire. TRUNCATE, commande DDL, ne peut pas être annulé.

Exercices

EXERCICES - BASE DU CENTRE

Base des exercices : centre de formation

Créer la base avec le script centre_formation_setup.sql, puis le relancer avant chaque exercice.

```
formation (id_formation, code, intitule, duree, niveau)
formateur (id_formateur, nom, prenom, email)
apprenant (id_apprenant, nom, prenom, email)
session (id_session, date_debut, date_fin, #id_formation, #id_formateur)
inscription (#id_apprenant, #id_session, date_inscription)
```

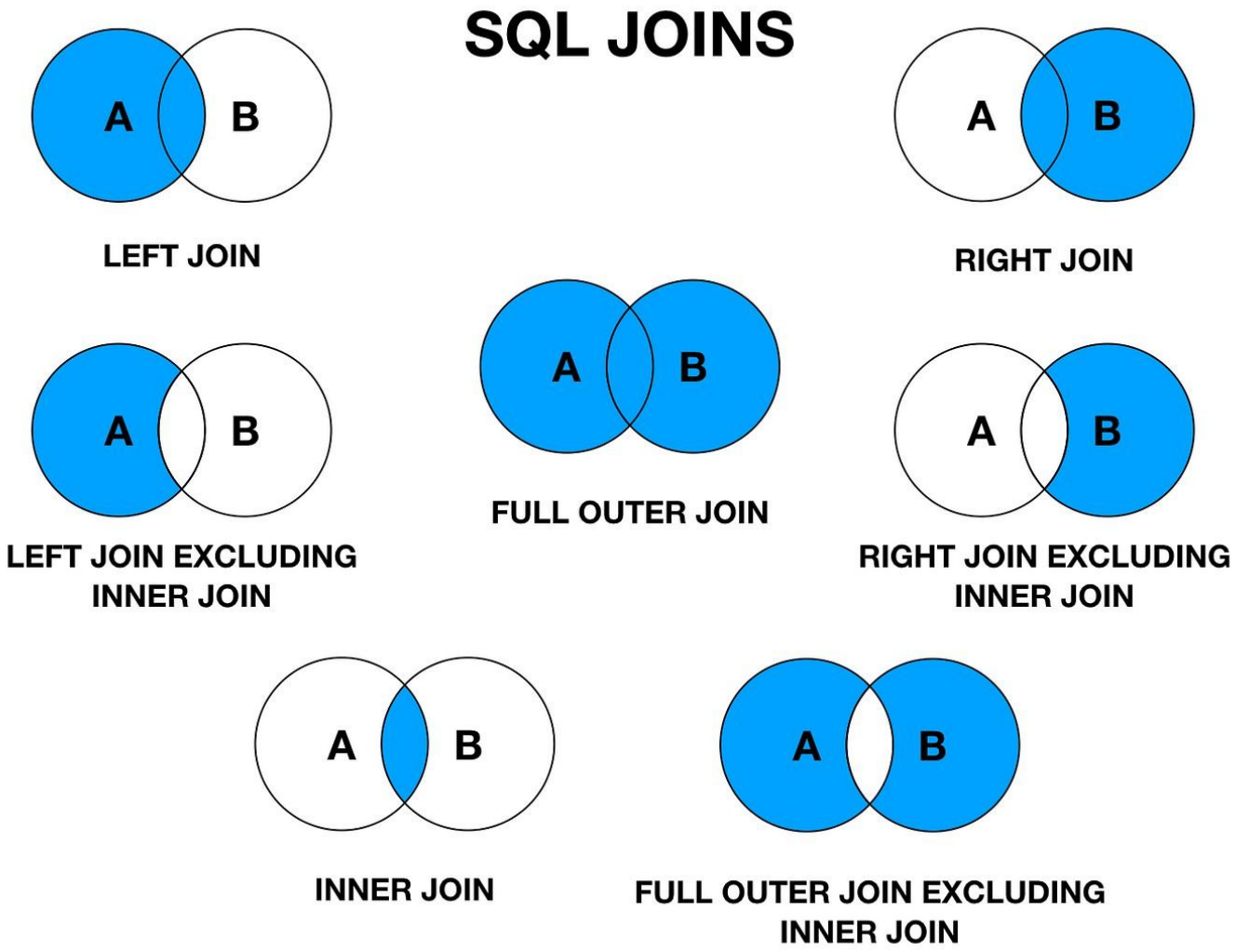
= clé étrangère. Une session correspond à une formation et à un formateur ; un apprenant s’inscrit à plusieurs sessions.

EXERCICE 1 - MANIPULATION

Écrire les requêtes suivantes

1. Ajouter la formation « SQL avancé » (code SQL-AVA, 21 h, niveau Avancé).
2. Inscrire l’apprenant 3 à la session 2, avec la date du jour.
3. Passer la durée de la formation PY-INI à 40 h.
4. Lister les formations de niveau Débutant, de la plus longue à la plus courte.
5. Lister les apprenants dont le nom commence par M.
6. Supprimer l’inscription de l’apprenant 7 à la session 4.

LES JOINTURES



LES JOINTURES

Introduction aux Jointures

Objectif :
Permettre de **lier des données** provenant de plusieurs tables pour répondre à des requêtes complexes.

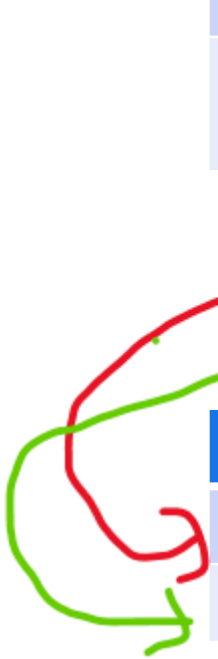
Pourquoi utiliser des jointures ?

- Une base bien conçue est **relationnelle** : les données sont réparties dans plusieurs tables.
- Les jointures permettent de **reconstituer l'information complète** à partir de ces tables liées.

Introduction aux Jointures

FILM

titre	annee	Id_realisateur	resume	Code_pays
Coco	2017	8	Depuis déjà...	EN
Le seigneur …	2001	108	Le jeune hobbit...	EN



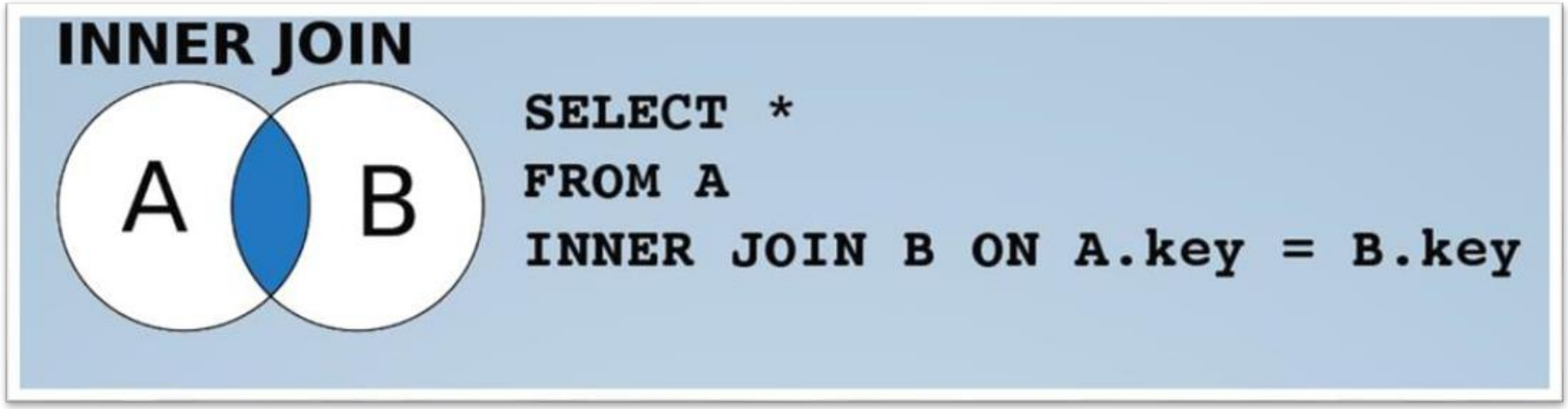
ARTISTE

id	nom	prenom	annee_naissance
8	Unkrich	Lee	1967
108	Jackson	Peter	1961

LES JOINTURES

INNER JOIN – Jointure interne

Retourne les lignes **avec correspondance dans les deux tables**.



LES JOINTURES

INNER JOIN – Jointure interne

Retourne les lignes **avec correspondance dans les deux tables**.

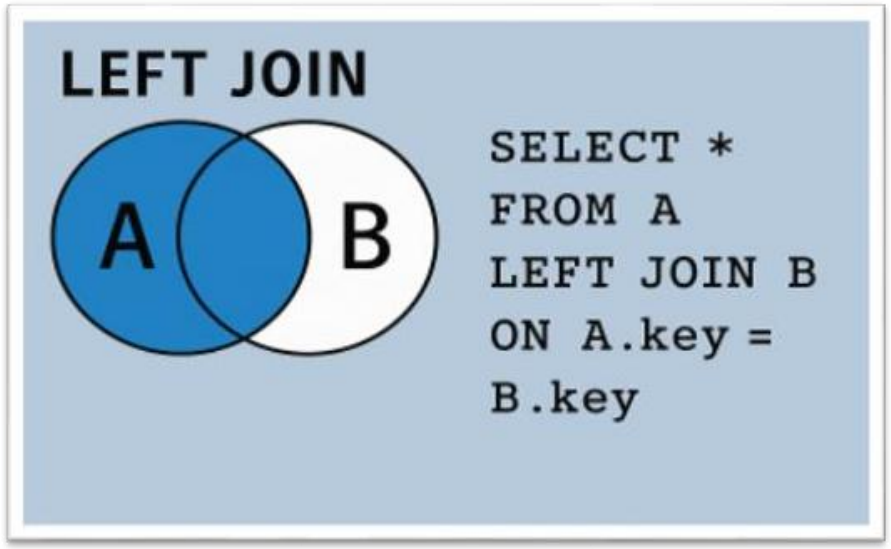
```
SELECT film.titre, artiste.nom
FROM film
INNER JOIN artiste
ON film.id_realisateur = artiste.id;
```

titre	nom
Coco	Unkrich
Le Seigneur des anneaux : La Communauté de l'annea...	Jackson
Le Seigneur des anneaux : Le Retour du roi	Jackson
La Liste de Schindler	Spielberg
Inception	Nolan

Affiche uniquement les films qui ont un réalisateur existant.

LEFT JOIN – Jointure gauche

Retourne **toutes les lignes de la table de gauche** (même sans correspondance dans la table de droite).



LEFT JOIN – Jointure gauche

Retourne **toutes les lignes de la table de gauche** (même sans correspondance dans la table de droite).

```
SELECT film.titre, artiste.nom
FROM film
LEFT JOIN artiste
ON film.id_realisateur = artiste.id;
```

titre	nom
Anastasia	*NULL*
Cosmic Chaos	*NULL*
Puri for Rent	*NULL*
Coco	Unkrich
Le Seigneur des anneaux : La Communauté de l'annea...	Jackson
Le Seigneur des anneaux : Le Retour du roi	Jackson
La Liste de Schindler	Spielberg

Affiche tous les films, même ceux **sans réalisateur défini** (NULL si aucun).

LEFT JOIN – sans *intersection*

Retourne **toutes les lignes de la table de gauche** qui n'ont pas de correspondance dans la table de droite.

```
SELECT film.titre, artiste.nom
FROM film
LEFT JOIN artiste
ON film.id_realisateur = artiste.id
WHERE artiste.id IS NULL;
```

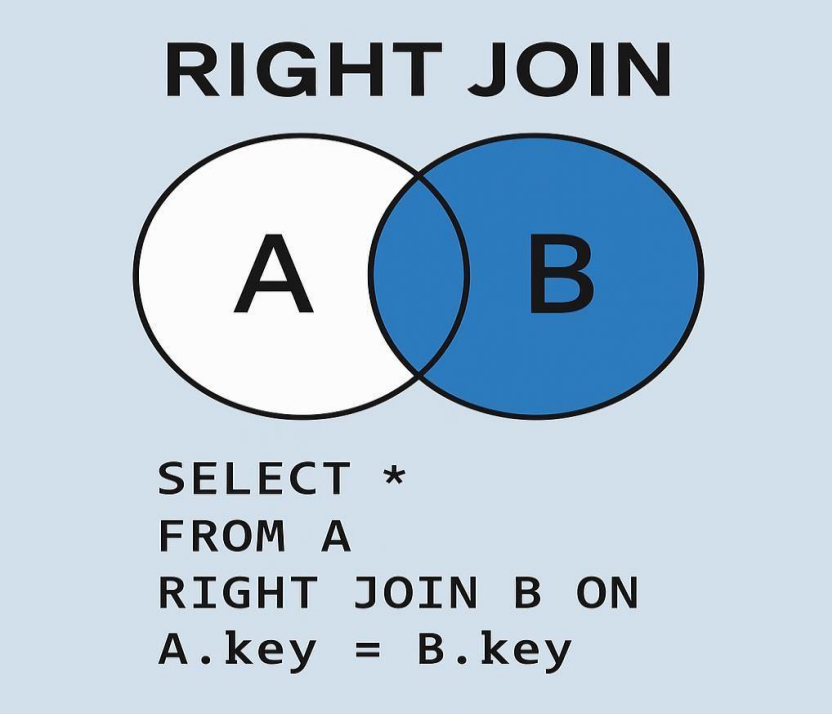
titre	nom
Anastasia	*NULL*
Cosmic Chaos	*NULL*
Puri for Rent	*NULL*
[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

Affiche les films qui n’ont **aucun réalisateur correspondant** (NULL).

LES JOINTURES

RIGHT JOIN – Jointure droite

Retourne toutes les lignes de la **table de droite**, même sans correspondance à gauche.



Joindre plus de deux tables

Une association N-N passe par une table de liaison : on enchaîne les JOIN le long des clés étrangères.

```
SELECT a.nom, f.intitule
FROM apprenant AS a
INNER JOIN inscription AS i
ON i.id_apprenant = a.id_apprenant
INNER JOIN session AS s
ON i.id_session = s.id_session
INNER JOIN formation AS f
ON s.id_formation = f.id_formation;
```

Chaque ON relie une clé étrangère à la clé primaire qu’elle référence (voir le MLD).

Exercices

EXERCICE 2 - LES JOINTURES

Écrire les requêtes suivantes

1. Afficher pour chaque session l’intitulé de la formation, le nom du formateur et la date de début.
2. Afficher toutes les formations, même celles sans session.
3. Afficher uniquement les formations qui n’ont aucune session.
4. Afficher les apprenants inscrits à la session 1, avec leur date d’inscription.
5. Afficher chaque apprenant avec l’intitulé des formations qu’il suit.

LE TEMPS ET LES FONCTIONS TEMPORELLES

```sql
CREATE TABLE utilisateurs (
  id INT PRIMARY KEY,
  nom VARCHAR(100),
  date_naissance DATE,
  date_inscription DATETIME
);

INSERT INTO utilisateurs
 (nom, date_naissance, date_inscription)
 VALUES 'Ali', '1990-05-15', '2025-09-21 14:30:00');
```

Apprendre MySQL : Date & DateTime



```sql
SELECT nom,
  DATE(date_inscription) AS jour,
  TIME(date_inscription) AS heure
FROM utilisateurs
WHERE date_inscription >= '2025-01-01'
  AND date_inscription < '2025-12-31'
  ORDER BY date_inscription;
```

DATE

DATETIME

2025-09-21
14:30:00

2025-01-01
14:30:00



LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Introduction

- La gestion des données temporelles est cruciale (dates d'inscription, transactions, etc.).
- SQL propose des **types de données temporelles** et des **fonctions** pour manipuler le temps.

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Les types de données temporelles

Type	Description	Exemple
DATE	Une date sans heure	2025-01-31
TIME	Une heure sans date	14:30:00
DATETIME	Date et heure combinées	2025-01-31 14:30:00
TIMESTAMP	type temporel dont la gestion dépend du SGBD et de sa gestion du fuseau horaire.	2025-01-31 13:30:00
YEAR	L'année uniquement	2025

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Fonctions de manipulation de dates

- `NOW()` → Renvoie la date et l'heure actuelles
- `CURDATE()` → Renvoie la date actuelle
- `CURTIME()` → Renvoie l'heure actuelle
- `DATE_ADD(date, INTERVAL n DAY)` → Ajoute n jours
- `DATEDIFF(date1, date2)` → Différence en jours
- `EXTRACT(part FROM date)` → Extrait une partie de la date (YEAR, MONTH, etc.)

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Exemples pratiques

```sql
SELECT NOW(); -- 2026-10-01 17:30:00
SELECT CURDATE(); -- 2025-04-06
SELECT DATE_ADD('2025-04-01', INTERVAL 5 DAY); -- 2025-04-06
SELECT DATEDIFF('2025-04-10', '2025-04-01'); -- 9
SELECT EXTRACT(YEAR FROM '2025-04-06'); -- 2025
```

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Applications concrètes

Filtrer les inscriptions récentes :

```
SELECT * FROM utilisateur WHERE date_inscription >= CURDATE() - INTERVAL 30 DAY;
```

Statistiques mensuelles :

```
SELECT COUNT(*) FROM vente WHERE MONTH(date_vente) = 4;
```

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Fonction DATE_FORMAT en SQL

La fonction *DATE_FORMAT* permet de formater une date selon un modèle spécifique.

Syntaxe :

- DATE_FORMAT(date, 'format')

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Fonction DATE_FORMAT en SQL

Exemples :

- DATE_FORMAT(NOW(), '%d/%m/%Y') → 06/04/2025
- DATE_FORMAT(NOW(), '%W %d %M %Y') → Sunday 06 April 2025
- DATE_FORMAT(date_col, '%Y-%m-%d %H:%i:%s') → 2025-04-06 14:30:00

LE TEMPS ET LES FONCTIONS TEMPORELLES EN SQL

Fonction DATE_FORMAT en SQL

- Principaux codes de format :
- %d : Jour (01 à 31)
- %m : Mois (01 à 12)
- %Y : Année (4 chiffres)
- %W : Nom du jour
- %M : Nom du mois
- %H : Heure (00 à 23)
- %i : Minutes
- %s : Secondes

LES FONCTIONS D'AGRÉGATION EN SQL

Définition

Les **fonctions d'agrégation** permettent de **résumer ou regrouper des données** sur une colonne.

Elles sont souvent utilisées avec `GROUP BY` pour effectuer des calculs sur des groupes de lignes.

LES FONCTIONS D'AGRÉGATION EN SQL

Fonctions principales

Fonction	Description
COUNT()	Compte le nombre de valeurs (ou de lignes)
SUM()	Calcule la somme des valeurs
AVG()	Calcule la moyenne des valeurs
MIN()	Donne la plus petite valeur
MAX()	Donne la plus grande valeur

NB :
COUNT(*) : compte les lignes
COUNT(email) : ne compte pas les NULL

LES FONCTIONS D'AGRÉGATION EN SQL

Exemples

```sql
-- Nombre total de films
SELECT COUNT(*) FROM film;

-- Somme des années de sortie
SELECT SUM(annee) FROM film;

-- Moyenne des années de sortie
SELECT AVG(annee) FROM film;

-- Année la plus ancienne
SELECT MIN(annee) FROM film;

-- Année la plus récente
SELECT MAX(annee) FROM film;
```

LES FONCTIONS D'AGRÉGATION EN SQL

GROUP BY et HAVING

`GROUP BY` – Organisation des résultats

- Utilisé pour **regrouper les lignes** qui ont des valeurs identiques dans une ou plusieurs colonnes.
- Souvent combiné avec des fonctions d'agrégation (`COUNT`, `SUM`, `AVG`, etc.).

LES FONCTIONS D'AGRÉGATION EN SQL

GROUP BY et HAVING

Exemple :

```sql
-- Nombre de films par pays
SELECT code_pays, COUNT(*) AS total
FROM film
GROUP BY code_pays;
```

LES FONCTIONS D'AGRÉGATION EN SQL

GROUP BY et HAVING

`HAVING` – Filtrage sur les groupes

- Contrairement à `WHERE` (qui filtre les lignes), **`HAVING` filtre les groupes** créés avec `GROUP BY`.

```sql
-- Pays ayant plus de 5 films
SELECT code_pays, COUNT(*) AS total
FROM film
GROUP BY code_pays
HAVING COUNT(*) > 5;
```

LES FONCTIONS D'AGRÉGATION EN SQL

GROUP BY et HAVING

Résumé :

Clause	Sert à...	S'utilise...
WHERE	Filtre les lignes individuelles	Avant le GROUP BY
HAVING	Filtre les groupes agrégés	Après le GROUP BY

Exercices

EXERCICE 3 - AGRÉGATION

Écrire les requêtes suivantes

1. Compter les apprenants.
2. Compter les inscrits de chaque session.
3. Afficher les sessions qui ont plus de 3 inscrits.
4. Calculer la durée moyenne des formations par niveau.
5. Calculer la durée en jours de chaque session.
6. Compter les inscriptions par mois (format AAAA-MM).

LES SOUS-REQUÊTES

Définition

Une **sous-requête** est une requête SQL imbriquée dans une autre requête. Elle permet d'extraire des données intermédiaires, souvent utilisées comme critères dans la requête principale.

Types de sous-requêtes :

- **Dans le `WHERE`** : pour filtrer selon un résultat dynamique.
- **Dans le `FROM`** : pour créer une table temporaire.
- **Dans le `SELECT`** : pour afficher une valeur issue d'une autre requête.

LES SOUS-REQUÊTES

Sous-requête dans WHERE

Structure :

```sql
SELECT nom
FROM produit
WHERE prix > (
    SELECT AVG(prix)
    FROM produit
);
```

On sélectionne les produits dont le prix est supérieur **au prix moyen de tous les produits**.

Sous-requête dans SELECT

```sql
SELECT nom,
       (SELECT AVG(note)
        FROM notation
        WHERE notation.id_film = film.id) AS moyenne
FROM film;
```

Cas typique :
Afficher une **valeur calculée spécifique** à chaque ligne.

LES SOUS-REQUÊTES

Sous-requête dans FROM (table dérivée)

Le résultat de la sous-requête sert de table temporaire, qui doit avoir un alias.

```
SELECT t.code_pays, t.total
FROM (SELECT code_pays, COUNT(*) AS total
      FROM film GROUP BY code_pays) AS t
WHERE t.total > 5;
```

Cas typique : filtrer ou joindre sur un résultat déjà agrégé.

Les clauses IN, ANY, ALL, EXISTS

IN – Appartenance à une liste ou un sous-ensemble

Syntaxe :

```
SELECT * FROM film WHERE code_pays IN ('FR', 'US');
```

Utilité : Vérifie si une valeur figure dans une liste donnée ou retournée par une sous-requête.

ANY – Comparaison avec au moins une valeur de la sous-requête

Syntaxe :

```
SELECT * FROM film
WHERE annee > ANY (
    SELECT annee FROM film WHERE code_pays = 'US'
);
```

Utilité : La condition est vraie si elle est vraie pour **au moins une** des lignes retournées.

Les clauses IN, ANY, ALL, EXISTS

`ALL` – Comparaison avec toutes les valeurs d'une sous-requête

Syntaxe :

```
SELECT * FROM film
WHERE annee > ALL (
    SELECT annee FROM film WHERE code_pays = 'FR'
);
```

Utilité : La condition est vraie **seulement si elle est vraie pour toutes les valeurs** retournées.

`EXISTS` – Vérifie l'existence de résultats d'une sous-requête

Syntaxe :

```
SELECT titre FROM film f
WHERE EXISTS (
    SELECT * FROM notation n WHERE n.id_film = f.id
);
```

Utilité : Retourne vrai si **au moins une ligne** est retournée par la sous-requête.

Exercices

EXERCICE 4 - SOUS-REQUÊTES

Écrire les requêtes suivantes

1. Afficher les formations plus longues que la durée moyenne.
2. Afficher les apprenants inscrits à au moins une session.
3. Afficher les apprenants qui n’ont aucune inscription.
4. Afficher les formateurs qui animent une session de la formation PY-INI.
5. Afficher chaque formation avec son nombre de sessions (sous-requête dans le SELECT).

ACCÈS AUX DONNÉES



C11 - CE QUE LE JURY ATTEND

Requêtes et composants d’accès aux données

Épreuve E1 : à partir d’un besoin exprimé, le candidat fournit :

- les requêtes nécessaires au besoin fonctionnel
- le code des composants d’accès aux données

Critères d’évaluation :

- l’ensemble des requêtes nécessaires sont fournies
- l’exécution des requêtes ne retourne aucune erreur
- le code permet l’accès à l’ensemble des requêtes
- l’exécution du code retourne les données attendues

Du SQL au code : se connecter à la base

Exemple en Node.js avec le module mysql2 :

```
const mysql = require("mysql2/promise");

async function main() {
  const conn = await mysql.createConnection({
    host: "localhost", user: "app_centre",
    password: process.env.DB_PASSWORD, database: "centre",
  });
  const [lignes] = await conn.execute(
    "SELECT intitule, duree FROM formation WHERE niveau = ?", ["Débutant"]);
  lignes.forEach((l) => console.log(l.intitule, l.duree));
  await conn.end();
}
main();
```

Ne jamais écrire le mot de passe en dur : variable d'environnement (process.env) ou fichier .env.

Requêtes paramétrées : éviter l'injection SQL

À ne jamais faire : concaténer une saisie utilisateur dans la requête.

```
const niveau = req.query.niveau;
await conn.query(`SELECT * FROM formation WHERE niveau = '${niveau}'`);
```

Avec la saisie x' OR '1'='1 la condition est toujours vraie : toutes les lignes sont renvoyées.

À faire : une requête paramétrée. La base traite la valeur comme une donnée, jamais comme du SQL.

```
await conn.execute("SELECT * FROM formation WHERE niveau = ?", [niveau]);
```

ACCÈS - ÉCRITURE

Persister les données et gérer les erreurs

Avec mysql2, l'autocommit est actif : chaque requête est validée tout de suite. Pour grouper plusieurs requêtes : beginTransaction(), puis commit(), ou rollback() en cas d'erreur.

```
try {
  await conn.beginTransaction();
  await conn.execute("DELETE FROM inscription " +
    "WHERE id_apprenant = ? AND id_session = ?", [3, 1]);
  await conn.execute("INSERT INTO inscription " +
    "(id_apprenant, id_session, date_inscription) VALUES (?, ?, CURDATE())", [3, 2]);
  await conn.commit();
} catch (erreur) {
  await conn.rollback();
}
```

ACCÈS - REPOSITORY

Isoler les requêtes dans un composant dédié

```
Vue  <-  Contrôleur  ->  Modèle (entité + Repository)  ->  Base de données
```

• Le repository (ou DAO) regroupe toutes les requêtes d'une table : find_all, find_by_id, create, update, delete.

• Le contrôleur appelle le repository : il n'écrit jamais de SQL.

• Avantages : requêtes centralisées et réutilisables, code testable, changement de SGBD localisé.

REPOSITORY - EXEMPLE (1/2)

FormationRepository : lire les données

```
class FormationRepository {
  constructor(conn) {
    this.conn = conn;
  }

  async findAll() {
    const [lignes] = await this.conn.execute(
      "SELECT * FROM formation ORDER BY intitule");
    return lignes;
  }

  async findById(idFormation) {
    const [lignes] = await this.conn.execute(
      "SELECT * FROM formation WHERE id_formation = ?", [idFormation]);
    return lignes[0];
  }
}
```

REPOSITORY - EXEMPLE (2/2)

FormationRepository : écrire les données

```
// suite de la classe FormationRepository
async create(code, intitule, duree, niveau) {
  const [res] = await this.conn.execute(
    "INSERT INTO formation (code, intitule, duree, niveau) VALUES (?, ?, ?, ?)",
    [code, intitule, duree, niveau]);
  return res.insertId;
}

async updateDuree(idFormation, duree) {
  const [res] = await this.conn.execute(
    "UPDATE formation SET duree = ? WHERE id_formation = ?", [duree, idFormation]);
  return res.affectedRows;
}

async delete(idFormation) {
  const [res] = await this.conn.execute(
    "DELETE FROM formation WHERE id_formation = ?", [idFormation]);
  return res.affectedRows;
}
```

Avec mysql2, l'autocommit est actif : chaque requête est validée immédiatement.

Utiliser un ORM

ORM (Object-Relational Mapping) : une table devient une classe, une ligne devient un objet.

• SQL brut : contrôle total des requêtes, mais beaucoup de code répétitif et dépendant du SGBD.

• ORM (Sequelize, Prisma, TypeORM, Hibernate, Eloquent…) : moins de code, portable d'un SGBD à l'autre, requêtes paramétrées par défaut.

• Limites : requêtes complexes parfois peu lisibles, risque de requêtes inefficaces (problème N+1).

• Le référentiel accepte les deux : SQL, ou couche d'abstraction d'un ORM.

ACCÈS - ORM (EXEMPLE)

Exemple avec Sequelize

```
const { Sequelize, DataTypes } = require("sequelize");

const sequelize = new Sequelize("centre", "app_centre",
  process.env.DB_PASSWORD, { host: "localhost", dialect: "mysql" });

const Formation = sequelize.define("Formation", {
  idFormation: { type: DataTypes.INTEGER, primaryKey: true, field: "id_formation" },
  intitule: DataTypes.STRING(100),
  niveau: DataTypes.STRING(20),
}, { tableName: "formation", timestamps: false });

(async () => {
  const debutants = await Formation.findAll({ where: { niveau: "Débutant" } });
  debutants.forEach((f) => console.log(f.intitule));
})();
```

ACCÈS - TESTS

Prouver que les requêtes et le code fonctionnent

• Jeu de données connu : relancer le script de création avant chaque série de tests.

• Pour chaque requête : exécution sans erreur, puis comparaison avec le résultat attendu.

• Pour chaque méthode du repository : cas nominal, cas limite (id inexistant), cas d'erreur (doublon UNIQUE).

Exemple (Jest) :

```
test("findById renvoie undefined si l'identifiant n'existe pas", async () => {
  expect(await repo.findById(9999)).toBeUndefined();
});
```

Exercices

EXERCICE 5 - COMPOSANTS

Écrire des composants d’accès aux données

1. Écrire getFormationsParNiveau(niveau) avec une requête paramétrée.

2. Ajouter findByCode(code) à FormationRepository.

3. Corriger la faille d’injection SQL dans cet extrait :

```
await conn.query("SELECT * FROM apprenant WHERE nom = '" + nom + "'");
```

4. Écrire changerSession(idApprenant, idAncienne, idNouvelle) dans une transaction.

5. Écrire 3 tests pour create() : cas nominal, code déjà existant, durée manquante.

ATELIER FINAL - C11

Besoin : application du centre de formation

L’application doit permettre de :

- consulter le catalogue des formations, filtré par niveau
- afficher le planning des sessions avec la formation et le formateur
- inscrire un apprenant à une session
- consulter les inscrits d’une session
- afficher le nombre d’inscrits par formation

Travail demandé (en binôme) : fournir les requêtes SQL, un repository Node.js et les preuves d’exécution.

ATELIER FINAL - C11

Livrables attendus

- requetes.sql : une requête par fonctionnalité, exécutée sans erreur
- un repository par table utilisée : requêtes paramétrées, commit et rollback
- un jeu de tests avec les résultats attendus
- les sorties ou captures d’exécution qui prouvent que les données attendues sont retournées

MERCI

*Une bonne **base de données**, c'est comme une bonne **relation** : sans **clé**, ça ne marche pas.*